

Experiment No. 1

Name: Parth Vijay Gulhane

Roll No: 23CO043

Class: TE Comp A

Title

Implementation of BFS and DFS on an Undirected Graph

Objectives

- To understand the concept of graph traversal
- To study Breadth First Search (BFS) algorithm
- To study Depth First Search (DFS) algorithm
- To implement BFS using queue data structure
- To implement DFS using recursion
- To compare BFS and DFS traversal techniques

Problem Statement

Develop a program to implement Breadth First Search (BFS) and Depth First Search (DFS) algorithms for traversing all the vertices of an undirected graph. The DFS algorithm should be implemented using a recursive approach.

Theory

Graph

A graph is a non-linear data structure consisting of nodes (vertices) and edges. It is represented as: $G = (V, E)$ Where:

- V = set of vertices
- E = set of edges Graphs can be:
- Directed or Undirected
- Weighted or Unweighted

In this experiment, we use an undirected graph, where edges have no direction.

Graph Traversal

Graph traversal is the process of visiting each vertex in a graph exactly once. It is widely used in searching, pathfinding, and network analysis.

Two main traversal techniques:

- Breadth First Search (BFS)
- Depth First Search (DFS)

Breadth First Search (BFS)

BFS explores the graph layer by layer. It first visits all nodes at distance 1 from the source, then distance 2, and so on.

Step-by-Step Working:

- Start from node A
- Visit all adjacent nodes (B, C)
- Then visit neighbours of B and C (D, E, F)

Key Properties:

- Guarantees shortest path in unweighted graph
- Uses queue to maintain order
- Avoids revisiting nodes using a visited set

Time Complexity:

$O(b^d)$

Space Complexity:

$O(b^d)$ due to queue and visited list

Applications of BFS

- Shortest path finding in graphs
- Social network analysis (finding mutual connections)
- Web crawling
- GPS navigation systems
- Broadcasting in networks

Depth First Search (DFS)

DFS explores as deep as possible before backtracking.

Step-by-Step Working:

- Start from node A
- Move to B → then D
- Backtrack to B → move to E → F

- Backtrack → go to C

Key Properties:

- Does not guarantee shortest path
- Uses recursion (implicit stack)
- Efficient for exploring deep structures

Time Complexity:

$O(b^d)$

Space Complexity:

$O(d)$ due to recursion stack

Applications of DFS

- Solving puzzles (like maze, Sudoku)
- Cycle detection in graphs
- Topological sorting
- Pathfinding in games
- Backtracking problems

Difference Between BFS and DFS

Feature	BFS	DFS
Data Structure	Queue	Stack / Recursion
Approach	Level-wise	Depth-wise
Memory Usage	More	Less
Application	Shortest path	Backtracking

Algorithms:

BFS:

1. Initialize queue
2. Mark start visited
3. Enqueue start
4. While queue not empty:
 - Dequeue node
 - Visit neighbors

DFS:

1. Mark current node as visited
2. Print the node
3. For each adjacent node:
 - If not visited, call DFS recursively

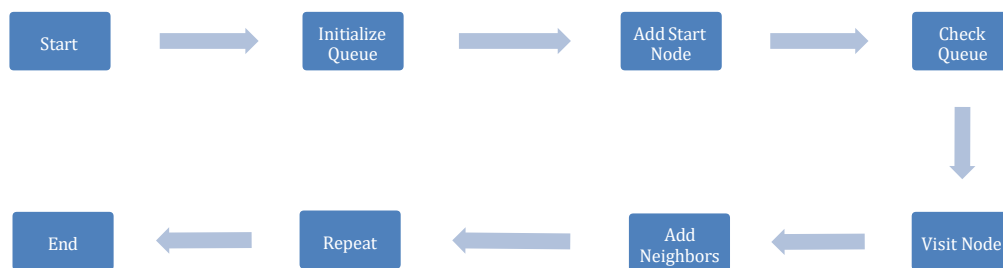
Mathematical Model:

$G = (V, E)$

$V = \{A, B, C, D, E, F\}$

Flowcharts

BFS Flowchart:



DFS Flowchart:



Output

```
DFS_BFS.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  class Graph
7  {
8      int V;
9      vector<vector<int>>
10     adj;
11 public:
12     Graph(int V){
13         this->V = V;
14         adj.resize(V);
15     }
16     void addEdge(int u, int v){
17         adj[u].push_back(v);
18         adj[v].push_back(u);
19     }
20     void DFSutil(int v, vector<bool> &visited){
21         visited[v] = true;
22         cout << v << " ";
23         for (int neighbor : adj[v]){
24             if (!visited[neighbor]){
25                 DFSutil(neighbor, visited);
26             }
27         }
28     }
29     void DFS(int start){
30         vector<bool> visited(V, false);
31         cout << "DFS Traversal: ";
32         DFSutil(start, visited);
33         cout << endl;
34     }
35     void BFS(int start){
36         vector<bool> visited(V, false);
37         queue<int> q;
38         visited[start] = true;
39         q.push(start);
40         cout << "BFS Traversal: ";
41         while (!q.empty()){
42             int v = q.front();
43             q.pop();
44             cout << v << " ";
45             for (int neighbor : adj[v]){
46                 if (!visited[neighbor]){
47                     visited[neighbor] = true;
48                     q.push(neighbor);
49                 }
50             }
51         }
52         cout << endl;
53     }
54 };
55
56 int main(){
57     Graph g(6);
58     g.addEdge(0, 1);
59     g.addEdge(0, 2);
60     g.addEdge(1, 3);
61     g.addEdge(1, 4);
62     g.addEdge(2, 5);
63     g.DFS(0);
64     g.BFS(0);
65     return 0;
66 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER

```
PS C:\Users\parth\Desktop\AI\Assignment\Assignment 1> cd "c:\Users\parth\Desktop\AI\Assignment\Assignment 1\" ; if ($?) { g++ DFS_BFS.cpp -o DFS_BFS } ; if ($?) { .\DFS_BFS }
PS C:\Users\parth\Desktop\AI\Assignment\Assignment 1>
```